

ShopEZ Trading Platform

1. Introduction

Project Title: ShopEZ Trading Platform

Lead Developer: Saurav Kumar

2. Project Overview

Purpose

The ShopEZ Trading Platform is a risk-free, real-time virtual stock trading simulator explicitly designed for novice investors. It aims to lower the barrier to entry for learning about the stock market by providing users with a virtual \$100,000 starting balance to practice trading, build financial confidence, and understand market dynamics without any real-world financial risk.

Key Features

- **Authentication Flow:** Secure User Login and Registration with role-based access control and automatic login redirection.
- **Live Market Explorer:** A real-time search interface to look up live stock symbols (e.g., AAPL, TSLA), view live open/high/low prices, and execute virtual "Buy" and "Sell" transactions.
- **User Dashboard:** Automatically initializes with \$100,000. Features a responsive table of current holdings alongside an interactive Pie Chart mapping exact asset allocation utilizing the Recharts library.
- **Admin Control Room:** A restricted, VIP route allowing users with the ADMIN database role to view a complete, centralized log of platform-wide trades.

3. Architecture

- **Frontend:** Built with React and Vite for fast compilation. It utilizes React Router for client-side routing, Bootstrap for responsive UI styling, and Recharts for complex data visualization. The frontend runs on Port 5173.
- **Backend:** Built with Node.js and Express.js. It acts as a stateless API server handling routing, JWT-based authentication validation, and acts as a proxy for fetching live stock market data. The backend runs on Port 5000.
- **Database:** MongoDB is utilized as the primary NoSQL data store, managed locally via MongoDB Compass. It securely stores user profiles, active portfolios (including current balances and stock holdings), and global transaction logs. The database runs on Port 27017.

4. Setup Instructions

Prerequisites

- Node.js (v16 or higher recommended)
- MongoDB Community Server and MongoDB Compass
- Git

Installation

1. Clone the repository to your local machine.
2. Navigate to the server directory and run `npm install` to install backend dependencies.
3. Navigate to the client directory and run `npm install` to install frontend dependencies.
4. Create a `.env` file in the server directory and configure your MongoDB URI (`mongodb://127.0.0.1:27017/shopez`) and JWT Secret keys.

5. Folder Structure

- **Client (Frontend):** Contains the React application components (e.g., Market.jsx, Portfolio.jsx, Admin.jsx), context providers for global state, assets, and Vite configuration files.
- **Server (Backend):** Organized into controllers (business logic), models (Mongoose schemas), routes (Express endpoints), and middleware (JWT authentication layers).

6. Running the Application

To run the application locally, start both the backend and frontend servers in separate terminal instances.

- **Backend:** Navigate to the server directory and run:
`npm start`
- **Frontend:** Navigate to the client directory and run:
`npm run dev`

Note for Windows environments: If you encounter PowerShell script block errors preventing execution, update the execution policy to RemoteSigned.

7. API Documentation

Endpoint	Method	Description	Parameters / Body
/api/auth/register	POST	Registers a new user and provisions \$100k balance.	{ email, password }
/api/auth/login	POST	Authenticates user and returns JWT token.	{ email, password }
/api/market/search	GET	Fetches live market data for a given symbol.	?symbol=AAPL
/api/portfolio	GET	Retrieves the current user's balance and stock holdings.	Requires JWT Authorization Header
/api/admin/logs	GET	Retrieves platform-wide trade logs (Admin only).	Requires JWT Authorization Header

8. Authentication

Authentication is handled securely using JSON Web Tokens (JWT). Upon successful login or registration, the backend issues a signed JWT to the client. The client must include this token in the Authorization header of subsequent requests. The backend middleware verifies the token and decodes the user's ID and role (User vs. Admin) to enforce role-based access control across protected routes.

9. User Interface

The UI is designed to be clean and intuitive, eliminating the overwhelming data noise typical of standard trading platforms.

- **Portfolio Dashboard:** Central hub displaying the virtual cash balance and a visual breakdown of asset allocation via an interactive pie chart.
- **Market Explorer:** A streamlined search tool offering clear "Buy" and "Sell" action buttons tied directly to real-time market valuations.

10. Testing

The testing strategy encompasses verifying critical user flows and database state management:

- **TC-001 (Initialization):** Verified that new user profiles generate with exactly \$100,000.
- **TC-002 (Live Trade):** Verified that executing a trade accurately updates the portfolio balance and holdings table.
- **TC-003 (Admin Route):** Verified that standard user accounts are blocked from accessing the Admin Control Room route.
- **TC-004 (Visuals):** Verified that the Recharts pie chart accurately maps holding ratios.

11. Screenshots or Demo

[Placeholders for UI Screenshots: Insert images of Portfolio.jsx, Market.jsx, and Admin.jsx here prior to final submission.]

12. Known Issues & Resolved Bugs

- **Resolved - Database Connectivity:** Diagnosed and fixed ECONNREFUSED 127.0.0.1:27017 errors by ensuring the MongoDB Windows background service is active before startup.

- **Resolved - PowerShell Execution:** Bypassed Windows security blocks preventing npm scripts by modifying execution policies to RemoteSigned.
- **Resolved - UI Graceful Fallbacks:** Addressed React bracket parsing errors in the Vite compiler and implemented graceful 404 error handling to prevent hard crashes and maintain a clean user dashboard.

13. Future Enhancements

- Integration of additional asset classes, such as Cryptocurrency and Forex.
- Implementation of advanced predictive analytics or machine learning modules for stock recommendations.
- Addition of multiple custom portfolios per user and deeper educational resources.